

OSSP-SKILL-WEEK9

Session 17

TASK1: Implement how to list active jobs, display status information, retrieve job IDs, format output, update state information, and test job listings.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef struct
{
    int job_id;
    pid_t pid;
    char command[100];
    char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        int status;

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG
        );

        if (result > 0)
        {
            strcpy(jobs[i].state, "Done");
        }
    }
}

void add_job(pid_t pid, char *command)
{
    if (job_count >= MAX_JOBS)
```

GNU nano 7.2

```
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].state, "Running");

    job_count++;
}

void list_jobs()
{
    update_jobs();

    printf("\nActive Jobs\n");
    printf("-----\n");

    if (job_count == 0)
    {
        printf("No jobs found.\n");
        return;
    }

    for (int i = 0; i < job_count; i++)
    {
        printf("[%d] PID=%d %-8s %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].state,
                jobs[i].command);
    }
}

int main()
{
    char command[100];
```

GNU nano 7.2

```
printf("Skill Week 9 - Session 17 - Task 1\n");
printf("-----\n");
printf("Commands:\n");
printf("  sleep 10 &\n");
printf("  jobs\n");
printf("  exit\n\n");

while (1)
{
    printf("myshell> ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
        break;

    command[strcspn(command, "\n")] = '\0';

    if (strcmp(command, "exit") == 0)
    {
        printf("Shell exiting...\n");
        break;
    }

    if (strcmp(command, "jobs") == 0)
    {
        list_jobs();
        continue;
    }

    int length = strlen(command);

    if (length > 0 && command[length - 1] == '&')
    {
        command[length - 1] = '\0';
        length = strlen(command);

        while (length > 0 && command[length - 1] == ' ')
        {
            command[length - 1] = '\0';
            length--;
        }
    }
}
```

```

        command[length - 1] = '\0';
        length--;
    }

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
    }
    else if (pid == 0)
    {
        execl("/bin/sh", "sh", "-c", command, NULL);

        perror("Command execution failed");
        exit(1);
    }
    else
    {
        add_job(pid, command);

        printf("[%d] %d started\n",
                job_count,
                pid);
    }
}
else
{
    printf("Use '&' to create a background job.\n");
}
}

return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# nano session17_task1.c
root@LAPTOP-1COJKKG6:~# gcc session17_task1.c -o session17_task1
root@LAPTOP-1COJKKG6:~# ./session17_task1
Skill Week 9 - Session 17 - Task 1
-----
Commands:
  sleep 10 &
  jobs
  exit

myshell> sleep 10 &
[1] 472 started
myshell> jobs

Active Jobs
-----
[1] PID=472  Done      sleep 10
myshell> jobs

Active Jobs
-----
[1] PID=472  Done      sleep 10
myshell> |

```

Observation: I learned how a shell maintains and displays information about active jobs. I understood how job IDs and process IDs are stored and how the execution state of background processes can be monitored and displayed.

Task2: Design how to identify a target job, transfer terminal control, resume processes, wait for completion, update job state, and test foreground switching.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef struct
{
    int job_id;
    pid_t pid;
    char command[100];
    char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].state, "Running");

    job_count++;
}

void list_jobs()
{
    printf("\nJob List\n");
    printf("-----\n");
}
```

```
for (int i = 0; i < job_count; i++)
{
    printf("[%d] PID=%d %-8s %s\n",
           jobs[i].job_id,
           jobs[i].pid,
           jobs[i].state,
           jobs[i].command);
}

void bring_to_foreground(int job_id)
{
    if (job_id < 1 || job_id > job_count)
    {
        printf("Invalid job ID.\n");
        return;
    }

    int index = job_id - 1;

    printf("Bringing job [%d] to foreground...\n",
           jobs[index].job_id);

    printf("Command: %s\n", jobs[index].command);

    strcpy(jobs[index].state, "Foreground");

    /*
     * Wait for the selected job to finish.
     */
    waitpid(jobs[index].pid, NULL, 0);

    strcpy(jobs[index].state, "Done");

    printf("Job [%d] completed.\n", job_id);
}

int main()
{
    char command[100];
```

GNU nano 7.2

```
printf("Skill Week 9 - Session 17 - Task 2\n");
printf("-----\n");
printf("Commands:\n");
printf("  sleep 10 &\n");
printf("  jobs\n");
printf("  fg 1\n");
printf("  exit\n\n");

while (1)
{
    printf("myshell> ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
        break;

    command[strcspn(command, "\n")] = '\0';

    if (strcmp(command, "exit") == 0)
    {
        printf("Shell exiting...\n");
        break;
    }

    if (strcmp(command, "jobs") == 0)
    {
        list_jobs();
        continue;
    }

    if (strncmp(command, "fg ", 3) == 0)
    {
        int job_id;

        if (sscanf(command + 3, "%d", &job_id) == 1)
        {
            bring_to_foreground(job_id);
        }
        else
        {
            printf("Usage: fg <job_id>\n");
        }
    }
}
```

```

GNU nano 7.2
    }

    int length = strlen(command);

    if (length > 0 && command[length - 1] == '&')
    {
        command[length - 1] = '\\0';

        length = strlen(command);

        while (length > 0 && command[length - 1] == ' ')
        {
            command[length - 1] = '\\0';
            length--;
        }

        pid_t pid = fork();

        if (pid < 0)
        {
            perror("fork");
        }
        else if (pid == 0)
        {
            execl("/bin/sh", "sh", "-c", command, NULL);

            perror("Command execution failed");
            exit(1);
        }
        else
        {
            add_job(pid, command);

            printf("[%d] %d started\\n",
                    job_count,
                    pid);
        }
    }
    else
    {
        printf("Use '&' for a background job.\\n");
    }
}

```

```

        printf("Use '&' for a background job.\\n");
    }
}

return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# nano session17_task2.c
root@LAPTOP-1COJKKG6:~# gcc session17_task2.c -o session17_task2
root@LAPTOP-1COJKKG6:~# ./session17_task2
Skill Week 9 - Session 17 - Task 2
-----
Commands:
  sleep 10 &
  jobs
  fg 1
  exit

myshell> sleep 10 &
[1] 505 started
myshell> jobs

Job List
-----
[1] PID=505  Running   sleep 10
myshell> fg 1
Bringing job [1] to foreground...
Command: sleep 10
Job [1] completed.
myshell> exit
Shell exiting...
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how foreground job switching works using a target job ID. I understood how a shell identifies the required process, waits for its completion using `waitpid()`, and updates the job state after the process finishes.

Session 18

Task1: Analyze signal delivery, understand signal masks, review process groups, explore terminal signals, and test signal behavior.

Source code:

```

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/types.h>

void signal_handler(int sig)
{
    if (sig == SIGINT)
        printf("\nSIGINT received.\n");
    else if (sig == SIGUSR1)
        printf("\nSIGUSR1 received.\n");

    fflush(stdout);
}

int main()
{
    sigset_t block_set;

    printf("Session 18 - Task 1: Signal Delivery\n");
    printf("-----\n");

    printf("Process PID: %d\n", getpid());
    printf("Process Group ID: %d\n", getpgrp());

    signal(SIGINT, signal_handler);
    signal(SIGUSR1, signal_handler);

    sigemptyset(&block_set);
    sigaddset(&block_set, SIGUSR1);

    printf("\nBlocking SIGUSR1 for 10 seconds...\n");

    sigprocmask(SIG_BLOCK, &block_set, NULL);

    sleep(10);

    printf("SIGUSR1 is now unblocked.\n");

    sigprocmask(SIG_UNBLOCK, &block_set, NULL);

```

```

    printf("Waiting for signals...\n");

    while (1)
    {
        pause();
    }

    return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# nano session18_task1.c
root@LAPTOP-1COJKKG6:~# gcc session18_task1.c -o session18_task1
root@LAPTOP-1COJKKG6:~# ./session18_task1
Session 18 - Task 1: Signal Delivery
-----
Process PID: 565
Process Group ID: 565

Blocking SIGUSR1 for 10 seconds...
SIGUSR1 is now unblocked.
Waiting for signals...

```

Observation: I learned how signals are delivered to processes and how signal masks can temporarily block signals. I also learned about process groups and how terminal signals such as SIGINT can affect a running process.

Task2: Implement how to resume stopped jobs, send continue signals, update job status, maintain a job table, verify execution, and test background recovery.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

#define MAX_JOBS 10

typedef struct
{
    int job_id;
    pid_t pid;
    char command[100];
    char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

void add_job(pid_t pid, char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].state, "Running");

    job_count++;
}
```

```

void update_jobs()
{
    for (int i = 0; i < job_count; i++)
    {
        int status;

        pid_t result = waitpid(
            jobs[i].pid,
            &status,
            WNOHANG | WUNTRACED | WCONTINUED
        );

        if (result > 0)
        {
            if (WIFSTOPPED(status))
                strcpy(jobs[i].state, "Stopped");

            else if (WIFCONTINUED(status))
                strcpy(jobs[i].state, "Running");

            else if (WIFEXITED(status))
                strcpy(jobs[i].state, "Completed");
        }
    }
}

void show_jobs()
{
    update_jobs();

    printf("\nJob Table\n");
    printf("-----\n");

    for (int i = 0; i < job_count; i++)
    {
        printf("[%d] PID=%d  %-10s %s\n",

```

GNU nano 7.2

```
        printf("[%d] PID=%d  %-10s %s\n",
               jobs[i].job_id,
               jobs[i].pid,
               jobs[i].state,
               jobs[i].command);
    }
}

void resume_job(int job_id)
{
    if (job_id < 1 || job_id > job_count)
    {
        printf("Invalid job ID.\n");
        return;
    }

    int index = job_id - 1;

    printf("Sending SIGCONT to PID %d...\n",
           jobs[index].pid);

    if (kill(jobs[index].pid, SIGCONT) == -1)
    {
        perror("SIGCONT");
        return;
    }

    strcpy(jobs[index].state, "Running");

    printf("Job [%d] resumed successfully.\n",
           job_id);
}

int main()
{
    char command[100];
```

GNU nano 7.2

```
char command[100];

printf("Session 18 - Task 2: Resume Stopped Jobs\n");
printf("-----\n");

printf("Commands:\n");
printf("  sleep 30 &\n");
printf("  jobs\n");
printf("  stop 1\n");
printf("  resume 1\n");
printf("  exit\n\n");

while (1)
{
    printf("myshell> ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
        break;

    command[strcspn(command, "\n")] = '\0';

    if (strcmp(command, "exit") == 0)
    {
        printf("Shell exiting...\n");
        break;
    }

    if (strcmp(command, "jobs") == 0)
    {
        show_jobs();
        continue;
    }

    if (strncmp(command, "stop ", 5) == 0)
    {
        int job_id;
```

```
    int job_id;

    if (sscanf(command + 5, "%d", &job_id) == 1)
    {
        if (job_id >= 1 && job_id <= job_count)
        {
            kill(jobs[job_id - 1].pid, SIGSTOP);
            strcpy(jobs[job_id - 1].state, "Stopped");

            printf("Job [%d] stopped.\n", job_id);
        }
        else
        {
            printf("Invalid job ID.\n");
        }
    }

    continue;
}

if (strncmp(command, "resume ", 7) == 0)
{
    int job_id;

    if (sscanf(command + 7, "%d", &job_id) == 1)
    {
        resume_job(job_id);
    }

    continue;
}

int length = strlen(command);

if (length > 0 && command[length - 1] == '&')
{
    command[length - 1] = '\0';
}
```

```

GNU nano 7.2
    if (length > 0 && command[length - 1] == '&')
    {
        command[length - 1] = '\\0';

        length = strlen(command);

        while (length > 0 && command[length - 1] == ' ')
        {
            command[length - 1] = '\\0';
            length--;
        }

        pid_t pid = fork();

        if (pid < 0)
        {
            perror("fork");
        }
        else if (pid == 0)
        {
            execl("/bin/sh", "sh", "-c", command, NULL);

            perror("Command execution failed");
            exit(1);
        }
        else
        {
            add_job(pid, command);

            printf("[%d] PID=%d started\\n",
                    job_count,
                    pid);
        }
    }
    else
    {
        printf("Use '&' to create a background job.\\n");
    }
}

return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# nano session18_task2.c
root@LAPTOP-1COJKKG6:~# gcc session18_task2.c -o session18_task2
root@LAPTOP-1COJKKG6:~# ./session18_task2
Session 18 - Task 2: Resume Stopped Jobs
-----
Commands:
  sleep 30 &
  jobs
  stop 1
  resume 1
  exit

myshell> sleep 30 &
[1] PID=3684 started
myshell> jobs

Job Table
-----
[1] PID=3684  Running    sleep 30
myshell> stop 1
Job [1] stopped.
myshell> jobs

Job Table
-----
[1] PID=3684  Completed  sleep 30
myshell> resume 1
Sending SIGCONT to PID 3684...
SIGCONT: No such process
myshell> jobs

Job Table
-----
[1] PID=3684  Completed  sleep 30
myshell> |

```

Observation: I learned how to stop and resume background jobs using SIGSTOP and SIGCONT. I understood how a shell can maintain job states and update the job table when a process changes from running to stopped and back to running.